Students:
Mostafa Ahmed [u6111119@udg.edu]
Houssam korib [u6106030@udg.edu]

Lab1: Robot Manipulation Tutorial: Stäubli Robotics Suite 2019 and VAL3

Nov 10, 2025

1 Task Description

The objective of this lab is to design, program, and implement a pick-and-place operation using the Stäubli TS60 SCARA robot in the Stäubli Robotics Suite (SRS) environment with the VAL3 programming language.

The task requires the robot to detect parts arriving on a conveyor, classify them based on their material type (metallic or non-metallic) using dedicated sensors, and place each part into the corresponding location within a predefined matrix.

Throughout this lab, multiple levels are introduced to progressively build the complete system:

- **Level 1:** Basic pick-and-place operation between two fixed points (from point A to point B) to understand robot motion programming in VAL3.
- **Level 2:** Integration of presence sensors and actuators to synchronize robot actions with conveyor movement and object detection.
- **Level 3:** Full implementation on the real TS60 robot with simultaneous control of the conveyor, classification sensors, barrier mechanism, and part counters for both metallic and non-metallic components.

By the end of this experiment, the robot autonomously performs the complete pick-and-place cycle: detecting a part, classifying it according to material, and placing it accurately in the correct matrix position until four metallic and four non-metallic parts are sorted successfully.

Program Breakdown

The developed VAL3 program controls the Stäubli TS60 SCARA robot to perform a fully automated pick-and-place operation integrated with sensors, actuators, and a conveyor system. The robot uses a sequence of joint movements (`MoveJ`) combined with digital I/O operations for gripper and conveyor control. The program continuously monitors multiple input sensors to detect, classify, and sort metallic and non-metallic parts into designated matrix positions.

Note: During implementation, the naming convention for points was reversed. The variables `place1` and `place2` correspond to the *pick points* (where the parts are grasped), while `pick1-pick8` represent the *placement points* in the classification matrix. This was done to remain consistent with the predefined simulation environment in Stäubli Robotics Suite.

Motion Sequence

The robot movement sequence is structured around a set of predefined points:

- **Up and Pick Points:** (`up1`, `pick1`, `up3`, `pick2`, etc.) – define safe approach and grasping positions.
- **Place Points:** (`place1`, `place2`) – define deposit positions for each part type.
- **Matrix Points:** (`pick1-pick8`) – represent matrix locations where metallic and non-metallic parts are placed based on classification.

Each movement command is executed using `MoveJ`, followed by `waitEndMove()` to ensure synchronization between the robot motion and the I/O operations.

I/O Operations

- **Outputs:**
  - `pick` – Controls gripper closing to pick a part.
  - `place` – Controls gripper opening to release a part.
  - `trolley` – Activates or stops the conveyor belt.
  - `barrier` – Raises or lowers the barrier to stop or release the part flow.
  - `metallic` / `nonmetallic` – Indicator signals for material classification.
- **Inputs:**
  - `Presenceclasify` – Detects part presence at classification point.
  - `metallicsensor` – Classifies the detected part as metallic or non-metallic.
  - `partPresence`, `rampPresence`, `Presencesuplied` – Detect part positions across the conveyor system.
  - `extended`, `contracted`, `extended1`, `contracted1` – Feedback sensors for pneumatic actuators.

Waiting and Synchronization

To ensure safe and reliable operation, short delays (typically `delay(0.5)` seconds) are introduced between consecutive I/O actions such as opening/closing the gripper, starting/stopping the conveyor, and activating the barrier. The `waitEndMove()` command guarantees that each motion finishes before proceeding to the next.

Program Flow (Pseudocode)

Algorithm 1 Pick and Place with Classification and Counting Logic

```
Initialize system and enable power
Set nominal robot speed (mNomSpeed.vel = 30)
Link I/O signals (dioLink())
Initialize counters: metalliccounter = 0, noncounter = 0
Display "System ready - waiting for parts"
while (metalliccounter < 4) OR (noncounter < 4) do
    Start conveyor (trolley = true)
    if Presenceclasify == true then
        Stop conveyor and raise barrier
        if metallicsensor == true then
            Set metallic output and reset nonmetallic output
            Move to pick position and close gripper
            Move to corresponding metallic matrix point (pick1, pick3, pick5, pick7)
            Open gripper and increment metalliccounter
        else
            Set nonmetallic output and reset metallic output
            Move to pick position and close gripper
            Move to corresponding nonmetallic matrix point (pick2, pick4, pick6, pick8)
            Open gripper and increment noncounter
        end if
    else
        Lower barrier and continue conveyor
    end if
    Short delay before next iteration (delay(0.5))
end while
Display "All parts processed successfully!"
Stop Routine:
Move to home position (jJointRs[0])
Deactivate suction, blow, trolley, and actuators
Cancel pending motion and display completion message
```

Task Parameters

The parameters used in this task include robot speeds, joint positions, Cartesian coordinates, tool frames, and payload settings. These parameters define the motion and behavior of the robot during the pick-and-place operation.

Joint Positions

The joint configurations correspond to the specific pick and place positions used in the program. The robot moves between several key points defined in the workspace, which include picking points and classified matrix positions.

Cartesian Points

For this task, we used a total of 10 Cartesian points:

- 4 points for metallic parts in the classified matrix.
- 4 points for non-metallic parts in the classified matrix.
- 2 points for picking the parts after they descend from the ramp following classification.

These points define the target positions and approach positions required for the robot to safely and accurately pick and place each part. The data for these points were collected by manually moving the robot using the teach pendant to the desired locations and saving the coordinates.

Tool Frame

The tool frame defines the orientation and position of the suction pad relative to the robot flange, as well as the timing for activating and deactivating the vacuum. This ensures secure gripping and release of the parts.

Robot Speed

The nominal speed of the robot is set to:

$$mNomSpeed.vel = 30$$

This speed is used for all joint and linear movements in the pick-and-place routine. For safety, the first test is conducted at a lower speed, gradually increasing once the program operates correctly.

Digital I/O (DIO)

The robot program uses several digital inputs and outputs to interact with the environment:

- **Outputs (bOut):**
  - bOut0: controls vacuum suction for the tool.
  - bOut1: controls blow to release the part.
  - bOut2: controls the trolley movement.

  - bOut3, bOut4: control class actuators for part placement.
  - bOut5: controls the barrier.

- **Inputs (bIn):**
  - bIn0: detects part presence on the tool.
  - bIn1: detects part ready for classification.
  - bIn2: metallic sensor (inductive sensor) detection.
  - bIn3: detects if part has been supplied.
  - bIn4: detects part at ramp.
  - bIn5–bIn8: detect extended/contracted state of the actuators.

These inputs and outputs are linked to the program using `dioLink()` to control and monitor the robot and cell devices automatically.

Safety Considerations

The Stäubli TS60 robot operates at significant speeds and can pose safety risks if not handled properly. To avoid collisions, all trajectories and motions were first verified in the simulation environment before testing on the real robot. The stop on first collision and emergency stop features were enabled and tested to ensure immediate halting in case of danger.

Safe robot configurations, such as the home position and pick/place points, were defined carefully to prevent collisions with the cell elements. All tool frames, Cartesian points, and joint positions were checked to maintain safe distances from obstacles. Additionally, digital inputs and outputs, actuator limits, and the robot controller's interlocks were used to prevent unintended movements, ensuring safe operation throughout the pick-and-place tasks.

Execution Results

The robot successfully completed the full pick-and-place sequence, handling all 8 parts (4 metallic and 4 non-metallic) with correct classification. The robot moved between the pick positions at the bottom of the ramps and the corresponding positions in the classified matrix, performing the necessary suction (pick) and release (place) operations. No collisions or unexpected behavior were detected, and all digital I/O actions executed as intended. The cycle times varied depending on part type and position, with the overall process demonstrating reliable and repeatable performance.

Improvements

The task could be further optimized by:
1. **Reducing cycle time**: Adjusting joint and linear speeds while ensuring safe operation, and minimizing pauses between pick and place actions.

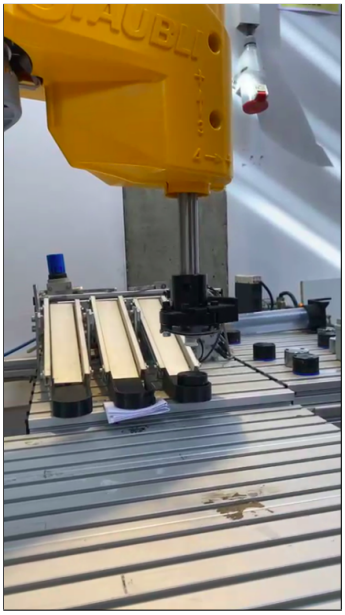
2. **Minimizing robot movements**: Reordering the sequence of metallic and non-metallic part handling to reduce travel distance across the matrix.
3. **Tool optimization**: Fine-tuning the suction pad positions and timings to reduce the duration of pick and place operations.
4. **Collision safety**: Refining the Cartesian points and trajectories to increase clearance from barriers, actuators, and other obstacles.
5. **Automation**: Integrating sensors and dynamic decision-making to detect part presence and adjust movements in real-time, reducing idle and waiting times.

Equipment setup

Images showcasing the setup for the application execution.



(a) Setup1



(b) Setup2

Figure 1: Application setup.